MEGN 651A: Homework Six

Rachel Bertaud

April 27th, 2026

The code used to produce this work can be located at this address:

https://github.com/rachbertaud/Advanced_CFD/tree/main/06_transient_advection_diffusion.

For reproduction of results, download the folder `06_transient_advection_diffusion` and enter the main directory. Simply run `$ make build` and `$ make run` to produce results for both spatial and temporal verification.

`$ make clean` removes all the executables and all the `txt\jpeg` files.

Overview of Code

The purpose of this code is to numerically solve unsteady, two-dimensional advection-diffusion equations at discrete locations using a finite-volume method with physical additions to the boundaries. Our goal is to verify the code both spatially and temporally.

Gov. Equations

The solute transport is governed by the steady advection-diffusion equation,

$$\int_{CV} \rho c_p \frac{\partial T}{\partial t}\, dV + \int_{CS} (\rho c_p \mathbf{u} T) \cdot \mathbf{n}\, dA = \int_{CS} k\, (\nabla T \cdot \mathbf{n})\, dA + \int_{CV} f(x,y,t)\, dV$$

for a steady, incompressible velocity field $\mathbf{u}(x,y)$ and constant thermophysical properties ρ, c_p, and k. We discretize the advection and diffusion terms using centered differences to maintain second-order spatial accuracy. Because the velocity field is steady, we discretize the problem using a fully-implicit backward Euler method. We use full Dirichlet boundary conditions.

Verification of Main

We first define our manufactured solution for the verification steps as

$$T_e(x,y,t) = \cos(x)\cos(y)\cos(\omega t), \tag{1}$$
$$u_e(x,y,t) = -\sin(x)\cos(y), \tag{2}$$
$$v_e(x,y,t) = \cos(x)\sin(y), \tag{3}$$

in the domain $L_x = L_y = 2\pi$, assuming $\rho = 1.1$, $c_p = 1.2$, and $k = 1.3$.

We apply the following Dirichlet boundary conditions;

$$T\Big|_{x=0,L_x} = \cos(y)\cos(\omega t), \qquad T\Big|_{y=0,L_y} = \cos(x)\cos(\omega t). \tag{4}$$

We build $\mathbf{A}$ and $\mathbf{b} = C_e$ (and fitting our BC) as explicitly defined in the overview section, and solve for $\mathbf{c}$. Using the results of this step, we verify that

1. Our solution's relative error measured with the infinity norm converges with order two accuracy and
2. Our operator, **A**, computes the expected solution for our governing equations.

The results of the verification process are defined below in the "Results" section.

Error Analysis

During the spatial verification process, error of the numerical solution is calculated as relative error using the infinity norm, which is defined as

$$E_\infty = \frac{||c_{\text{numerical}} - c_{\text{analytical}}||_\infty}{||c_{\text{analytical}}||_\infty} = \frac{\max|c_{\text{numerical}} - c_{\text{analytical}}|}{\max|c_{\text{analytical}}|}.$$

Note that we will also define local error as

$$\text{Local Error} = |c_{\text{numerical}} - c_{\text{analytical}}|.$$

We also define the residual as

$$\dot{T}_{\max} = \frac{\|T^{n+1} - T^n\|_\infty}{\Delta t}.$$

Results

Overall, the results of this code verified its capabilities. Everything performed as anticipated. Refer to

- Figure 1 — where we confirmed second-order spatial accuracy!
- Figure 2 — where we confirmed first-order temporal accuracy!

Problem 7.3 (Spatial Verification)

For spatial verification, we define initial conditions of

$$T\Big|_{t=0} = 0, \quad \omega = 0.$$

The results shown are for varying grid sizes marched in time until $\dot{T}_{\max} < 10^{-10}$. Then, the error between the numerical solution and analytical solution is computed as described by E_∞ above. Results are consistent with order 2 accuracy, which is what we expect!

Problem 7.4 (Temporal Verification)

For temporal verification, we define initial conditions of

$$T\Big|_{t=0} = \cos(x)\cos(y), \quad \omega = 2\pi.$$

The results shown are for varying time steps up marched from $t = 0$ t $t = 1$. Then, the error between the numerical solution and analytical solution is computed as described by E_∞ above. Results are consistent with order 1 accuracy, which is what we expect! Both plots also show an error floor that is dependent on the spatial resolution. Note that for $Nx = Ny = 64$ we see the floor appear later and carry a smaller error. This points out the coupling between spatial and temporal accuracy that this system relies on!

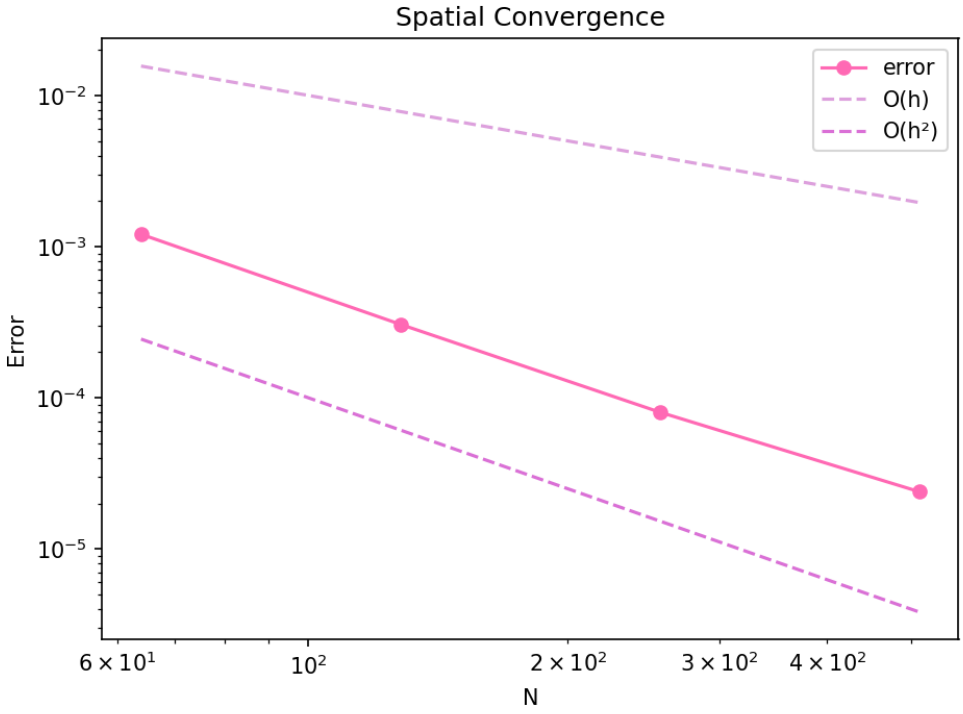
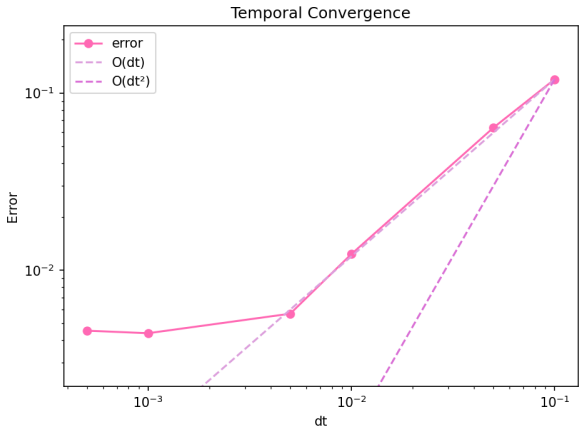
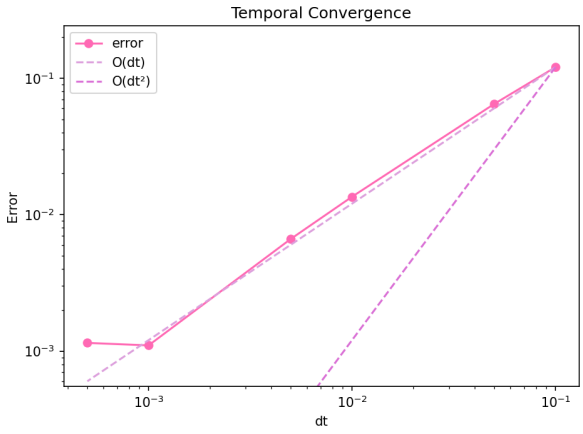


Figure 1: Relative ℓ_∞ error E_∞ of the numerical solution as a function of the number of grid points N, compared against $\mathcal{O}(h)$ and $\mathcal{O}(h^2)$ reference lines. The solution converges at second-order, consistent with the $\mathcal{O}(h^2)$ reference.



(a) Plot of temporal convergence for $Nx = Ny = 32$.



(b) Plot of temporal convergence for $Nx = Ny = 64$.

Figure 2: Relative ℓ_∞ error E_∞ of the numerical solution as a function of the temporal resolution Δt, compared against $\mathcal{O}(h)$ and $\mathcal{O}(h^2)$ reference lines. The solution converges at first-order, consistent with the $\mathcal{O}(h)$ reference. Note the floor bounded by spatial accuracy on both!

Code

For the sake of page usage, I am only including the main code for the temporal and spatial convergence tests. Everything else that was used to produce these results can be found at this GitHub location. Please refer to the first page for reproduction instructions.

$\texttt{main}_T E.cpp$

```
#include "Header_VolSF.H"
#include "Header_Mesh.H"
#include "Header_velF.H"
#include "Header_ScaSF.H"
#include "Header_matrix.H"
#include "Header_bcType.H"
#include "Header_Time.H"
#include "Header_SpatError.H"

int main()
{
  Time runTime;

  int flag = 0;

  meshType Mesh;

  volScalarField T("T", Mesh, runTime);
  T.setUniformIC(0, 1, Mesh);
  SpatError errorComp(T);


  velocityField U(Mesh);
  surfaceScalarField Flux(Mesh);
  Flux.ComputePhix(Mesh, T, U);
  Flux.ComputePhiy(Mesh, T, U);

  matrixType Matrix(Mesh);
  Matrix.BuildA(Mesh, Flux, T, runTime);
  Matrix.factorize();

  while( runTime.run() )
  {
    runTime.advance();

    T.setSource(Mesh, runTime);
    Matrix.BuildRHS(Mesh, T, U, runTime);

    Matrix.solve(T);


    if (runTime.writeNow())
    {
      T.compTmax(runTime);
      T.Save(runTime);
    }

    if ( std::fabs(runTime.time - 1.0) < 1e-7 )
    {
      errorComp.setAnSol(Mesh, runTime);
```

```
51        errorComp.comp(T);
52        break;
53      }
54
55      T.update();
56    }
57
58
59    return flag;
60  };
```

main$_SE.cpp$

```
1  #include "Header_VolSF.H"
2  #include "Header_Mesh.H"
3  #include "Header_velF.H"
4  #include "Header_ScaSF.H"
5  #include "Header_matrix.H"
6  #include "Header_bcType.H"
7  #include "Header_Time.H"
8  #include "Header_SpatError.H"
9
10 int main()
11 {
12   Time runTime;
13
14   int flag = 0;
15
16   meshType Mesh;
17
18   volScalarField T("T", Mesh, runTime);
19   T.setUniformIC(0, 0, Mesh);
20   SpatError errorComp(T);
21
22
23   velocityField U(Mesh);
24   surfaceScalarField Flux(Mesh);
25   Flux.ComputePhix(Mesh, T, U);
26   Flux.ComputePhiy(Mesh, T, U);
27
28   matrixType Matrix(Mesh);
29   Matrix.BuildA(Mesh, Flux, T, runTime);
30   Matrix.factorize();
31
32   while( runTime.run() )
33   {
34     runTime.advance();
35
36     T.setSource(Mesh, runTime);
37     Matrix.BuildRHS(Mesh, T, U, runTime);
38
39     Matrix.solve(T);
40
41
42     if (runTime.writeNow())
43     {
44       T.compTmax(runTime);
45       //T.Save(runTime);
46     }
```

```
47
48      if (errorComp.compStart(T, runTime) )
49      {
50        errorComp.setAnSol(Mesh, runTime);
51        errorComp.comp(T);
52        break;
53      }
54
55      T.update();
56    }
57
58
59    return flag;
60  };
```